Differential Drive Mobile Robot Simulation and Dead Reckoning

Diego Marzá Fuertes and Mario López Honrubia

October 2026

Abstract

This report presents the implementation and validation of a differential-drive mobile robot simulator and a dead-reckoning localisation method. Planar direct and inverse pose compounding were first implemented and then incorporated into the simulation and localisation modules. The simulator integrates the commanded linear and angular velocities while generating noisy wheel-encoder readings. These measurements are subsequently used to estimate the robot pose without access to the simulated ground truth. The resulting trajectories show the expected accumulation of odometric error: the position error is 1.019 m after the first 5000 simulation steps and reaches 12.481 m after the complete 10000-step sequence. The exercise demonstrates how small encoder errors propagate through successive motion updates and produce increasing drift in dead-reckoning localisation.

1 Introduction

The objective of Labs 1 and 2 is to build the simulation and odometric localisation foundations required by the later probabilistic localisation exercises. The Pre-Lab covered the planar pose representation and the direct and inverse pose-compounding operators. Lab 1 then applied these operations to the differential-drive simulator, wheel-encoder model, and commanded trajectories. Lab 2 used the simulated encoder readings to estimate the robot pose through dead reckoning.

The implementation uses a world frame, denoted by N, and a body-fixed robot frame, denoted by B. A planar robot pose contains two Cartesian coordinates and one yaw angle. Lab 1 produces the simulated or ground-truth state, whereas Lab 2 reconstructs the pose using only incremental encoder information.

The report is organised accordingly. The remainder of this introduction describes the software structure and the Pre-Lab pose operations. Section 2 presents the differential-drive simulation, and section 3 describes the dead-reckoning implementation. The experimental comparison is discussed in section 4, followed by the conclusions in section 5.

1.1 Development environment and software structure

Jupyter Notebook was used only in Lab 0 to implement and validate the direct and inverse pose-compounding operations, $\oplus$ and $\ominus$. These operations were subsequently incorporated into the Python modules used in Labs 1 and 2. NumPy provides vector and matrix operations, Matplotlib is used for visualisation, and Robotics Toolbox supplies the animated vehicle representation.

The relevant software components and their roles are summarised in table 1.

Table 1: Main Python files used in Labs 1 and 2.

File	Responsibility
`Pose3D.py`	Represents a planar pose and implements direct and inverse pose compounding.
`SimulatedRobot.py`	Defines the common simulation interface, state storage and visualisation infrastructure.
`DifferentialDriveSimulatedRobot.py`	Implements the differential-drive motion model and simulated sensors.
`Localization.py`	Provides the iterative localisation loop and trajectory visualisation.
`DR_3DOFDifferentialDrive.py`	Converts encoder readings into body motion and performs dead-reckoning integration.
`main.py`	Configures the map and initial state, creates the simulated and dead-reckoning robots, and runs the localisation sequence.

1.2 Pre-Lab: Planar pose operations

1.2.1 Planar pose representation

A three-degree-of-freedom pose is represented by

$$\boldsymbol{\eta} = \begin{bmatrix} x & y & \psi \end{bmatrix}^T \in \mathbb{R}^3, \tag{1}$$

where (x, y) is the position and ψ is the yaw angle. Its equivalent homogeneous transformation is

$$T(\boldsymbol{\eta}) = \begin{bmatrix} \cos\psi & -\sin\psi & x \\ \sin\psi & \cos\psi & y \\ 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

This matrix form explains the pose operations used throughout the simulator: direct compounding corresponds to multiplying transformations, and inverse compounding corresponds to inverting a transformation.

1.2.2 Direct pose compounding

Let ${}^{A}\boldsymbol{x}_B$ describe frame B relative to frame A, and let ${}^{B}\boldsymbol{x}_C$ describe frame C relative to B. The pose of C relative to A is

$$ {}^{A}\boldsymbol{x}_C = {}^{A}\boldsymbol{x}_B \oplus {}^{B}\boldsymbol{x}_C, \tag{3} $$

which is equivalent to $T({}^{A}\boldsymbol{x}_C) = T({}^{A}\boldsymbol{x}_B)T({}^{B}\boldsymbol{x}_C)$. In coordinate form,

$$ \begin{bmatrix} {}^{A}x_C \\ {}^{A}y_C \\ {}^{A}\psi_C \end{bmatrix} = \begin{bmatrix} {}^{A}x_B + {}^{B}x_C\cos({}^{A}\psi_B) - {}^{B}y_C\sin({}^{A}\psi_B) \\ {}^{A}y_B + {}^{B}x_C\sin({}^{A}\psi_B) + {}^{B}y_C\cos({}^{A}\psi_B) \\ {}^{A}\psi_B + {}^{B}\psi_C \end{bmatrix}. \tag{4} $$

The $\oplus$ operator is used to compose poses or motions expressed in different reference frames. In this case, ${}^{B}\boldsymbol{x}_C$ cannot be added directly to ${}^{A}\boldsymbol{x}_B$ because its components are written in frame B. The operation first transforms them into frame A and then combines both poses to obtain ${}^{A}\boldsymbol{x}_C$.

The implementation extracts the pose components, evaluates equation (4), and returns another `Pose3D` object. The essential operation is:

Listing 1: Core of the direct compounding implementation.

```
x_result = xB + xC * cos(theta_B) - yC * sin(theta_B)
y_result = yB + xC * sin(theta_B) + yC * cos(theta_B)
theta_result = theta_B + theta_C
return Pose3D(np.array([[x_result], [y_result], [theta_result]]))
```

1.2.3 Inverse pose compounding

Inverse compounding expresses frame A relative to frame B from the pose of B relative to A:

$$ {}^{B}\boldsymbol{x}_A = \ominus {}^{A}\boldsymbol{x}_B = \begin{bmatrix} -{}^{A}x_B\cos({}^{A}\psi_B) - {}^{A}y_B\sin({}^{A}\psi_B) \\ {}^{A}x_B\sin({}^{A}\psi_B) - {}^{A}y_B\cos({}^{A}\psi_B) \\ -{}^{A}\psi_B \end{bmatrix}. \tag{5} $$

This is the coordinate form of $T({}^{B}\boldsymbol{x}_A) = T({}^{A}\boldsymbol{x}_B)^{-1}$.

The implementation evaluates equation (5) directly and returns the inverse pose as another `Pose3D` object:

Listing 2: Core of the inverse compounding implementation.

```
x_result = -xB * cos(theta_B) - yB * sin(theta_B)
y_result =  xB * sin(theta_B) - yB * cos(theta_B)
theta_result = -theta_B
return Pose3D(np.array([[x_result], [y_result], [theta_result]]))
```

After implementing and testing both operators in the Pre-Lab notebook, the same code was included in `Pose3D.py`. This makes $\oplus$ and $\ominus$ available as `Pose3D` methods for the differential-drive simulation developed in Lab 1.

2 Lab 1: Simulation

Lab 1 consists of completing the `DifferentialDriveSimulatedRobot` class, which inherits from `SimulatedRobot`. The base class provides the common simulation interface, state storage, and visualisation tools, whereas the derived class implements the differential-drive motion and sensor models.

2.1 `__init__`: robot configuration

The constructor first calls `super().__init__` to initialise the state, map, sampling period, and visualisation variables provided by `SimulatedRobot`. The simulated state contains the world-frame pose and body-frame velocity:

$$\boldsymbol{x}_{s,k} = \begin{bmatrix} \boldsymbol{\eta}_{s,k} \\ \boldsymbol{\nu}_{s,k} \end{bmatrix} = \begin{bmatrix} x_k & y_k & \psi_k & u_k & v_k & r_k \end{bmatrix}^T. \tag{6}$$

It then defines the parameters required by the differential-drive model. The wheel separation is $b = 0.5$ m, the wheel radius is $R = 0.1$ m, and each wheel revolution produces $N_p = 1024$ encoder pulses. The motion and encoder noise are described by

$$Q_s = \text{diag}\left(0.1^2,\, 0.01^2,\, \text{deg2rad}(1)^2\right), \qquad R_e = \text{diag}(22^2, 22^2). \tag{7}$$

Here, Q_s is the covariance of the acceleration perturbation applied by the motion model, while R_e is the covariance of the encoder noise in squared pulses. The remaining constructor parameters configure sensors that are not required for the simulation and dead-reckoning tasks discussed here.

2.2 `fs`: motion model

This method computes the new simulated state from the previous state and the commanded forward and angular velocities. The desired input is $\boldsymbol{u}_{s,k} = [u_d,\, r_d]^T$. Because a differential-drive robot cannot move laterally, the corresponding desired body velocity is $\boldsymbol{\nu}_d = [u_d,\, 0,\, r_d]^T$.

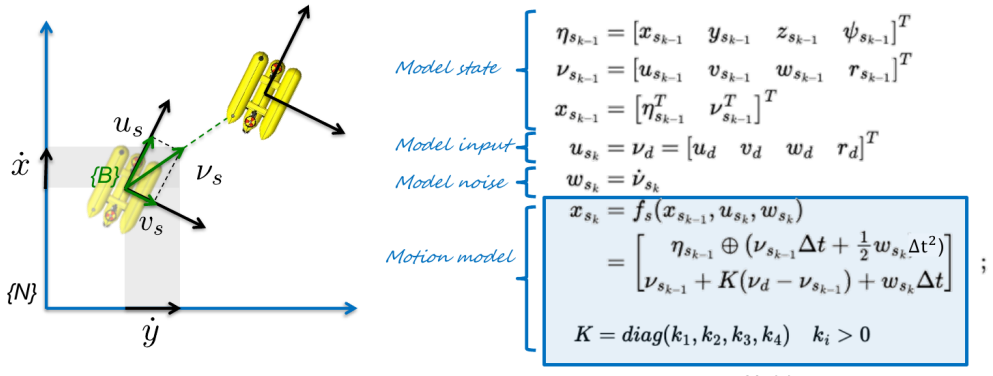


Figure 1: State, input, noise, and motion model used by `fs`.

The course model in figure 1 is written for four degrees of freedom. For the planar differential-drive robot, the vertical position and velocity are removed, giving $\boldsymbol{\eta} = [x,\, y,\, \psi]^T$ and $\boldsymbol{\nu} = [u,\, v,\, r]^T$. The same state-transition structure is retained, with $v_d = 0$ because the wheels do not permit lateral motion.

At each simulation step, the acceleration perturbation is sampled from the covariance configured by the constructor:

$$\boldsymbol{w}_{s,k} \sim \mathcal{N}(\mathbf{0}, Q_s), \qquad Q_s = \texttt{self.Qsk}. \tag{8}$$

With sampling period $\Delta t = 0.1$ s, the implemented update is

$$\Delta\boldsymbol{\eta}_k = \boldsymbol{\nu}_{s,k-1}\Delta t + \tfrac{1}{2}\boldsymbol{w}_{s,k}\Delta t^2, \tag{9}$$

$$\boldsymbol{\eta}_{s,k} = \boldsymbol{\eta}_{s,k-1} \oplus \Delta\boldsymbol{\eta}_k, \tag{10}$$

$$\boldsymbol{\nu}_{s,k} = \boldsymbol{\nu}_{s,k-1} + K(\boldsymbol{\nu}_d - \boldsymbol{\nu}_{s,k-1}) + \boldsymbol{w}_{s,k}\Delta t, \tag{11}$$

where the implementation uses $K = I_3$. Pose compounding rotates the body-frame increment into the world frame before adding it to the previous pose.

The key lines in `fs` are therefore:

Listing 3: Pose and velocity update in the simulator.

```
w = np.random.multivariate_normal(np.zeros(3), self.Qsk).reshape(3, 1)
eta_new = eta_prev.oplus(Pose3D(
    nu_prev * self.dt + 0.5 * w * self.dt**2))
nu_new = nu_prev + K @ (nu_desired - nu_prev) + w * self.dt
```

2.3 Wheel-encoder simulation

The encoder model first computes the motion between consecutive simulated poses in the previous body frame:

$$\Delta\boldsymbol{\eta}_k = \left(\ominus\boldsymbol{\eta}_{s,k-1}\right) \oplus \boldsymbol{\eta}_{s,k}. \tag{12}$$

Let $d = \Delta x_k$ be the local forward displacement and $\Delta\psi = \Delta\psi_k$ the yaw increment. For wheel separation $b = 0.5$ m, the left and right wheel displacements are

$$d_L = d - \frac{b}{2}\Delta\psi, \qquad d_R = d + \frac{b}{2}\Delta\psi. \tag{13}$$

For wheel radius $R = 0.1$ m and N_p pulses per revolution, distance is converted to pulses by

$$\begin{bmatrix} n_L \\ n_R \end{bmatrix}_{\text{ideal}} = \frac{N_p}{2\pi R}\begin{bmatrix} d_L \\ d_R \end{bmatrix}. \tag{14}$$

Independent zero-mean Gaussian measurement noise is then added using

$$R_e = \operatorname{diag}(22^2, 22^2)\ \text{pulses}^2. \tag{15}$$

The resulting vector $\boldsymbol{z}_{s,k} = [n_L, n_R]^T$ is the only motion information used by the dead-reckoning estimator.

2.4 Trajectory commands

The simulator is tested with a constant forward speed and opposite angular velocities. The first command produces circular motion; changing the angular velocity sign produces the second lobe of the requested figure-eight sequence:

Listing 4: Commands used for the trajectory tests.

```
dr_robot.LocalizationLoop(x0, np.array([[0.5,  0.03]]).T)
dr_robot.LocalizationLoop(dr_robot.xk, np.array([[0.5, -0.03]]).T)
```

The second run starts from the final estimate of the first run, preserving continuity between the two trajectory segments.

3 Lab 2: Dead-Reckoning Localization

3.1 Localisation sequence

The `Localization` base class defines the common iterative procedure. At every time step it:

1. advances the simulated robot through `robot.fs`;
2. obtains the encoder-derived input through `GetInput`;
3. updates the pose estimate through `Localize`; and
4. plots the estimated trajectory.

The derived `DR_3DOFDifferentialDrive` class supplies the two algorithm-specific methods in steps 2 and 3.

3.2 Encoder readings to body motion

The tested dead-reckoning configuration uses $N_p = 4096$ pulses per wheel revolution. Each encoder reading is first converted back to a travelled distance:

$$d_L = n_L \frac{2\pi R}{N_p}, \qquad d_R = n_R \frac{2\pi R}{N_p}. \tag{16}$$

The differential-drive displacement is then

$$d = \frac{d_L + d_R}{2}, \qquad \Delta\psi = \frac{d_R - d_L}{b}. \tag{17}$$

Dividing by the sampling period gives the body velocity used by the localisation model:

$$\boldsymbol{\nu}_k = \begin{bmatrix} u_k \\ v_k \\ r_k \end{bmatrix} = \begin{bmatrix} d/\Delta t \\ 0 \\ \Delta\psi/\Delta t \end{bmatrix}. \tag{18}$$

The most relevant implementation fragment is:

Listing 5: Conversion from encoder pulses to body velocity.

```
dist_per_pulse = (2 * np.pi * self.wheelRadius) \
                 / self.robot.pulse_x_wheelTurns
d_L, d_R = n_L * dist_per_pulse, n_R * dist_per_pulse
d = (d_L + d_R) / 2
theta = (d_R - d_L) / self.wheelBase
return np.array([[d / self.dt], [0.0], [theta / self.dt]])
```

3.3 Dead-reckoning pose update

The estimator integrates the encoder-derived velocity over one sampling period and compounds that body-frame displacement with the previous estimate:

$$\widehat{\boldsymbol{\eta}}_k = \widehat{\boldsymbol{\eta}}_{k-1} \oplus (\boldsymbol{\nu}_k \Delta t)\,. \tag{19}$$

The complete mathematical operation is represented by one central line in `Localize`:

Listing 6: Dead-reckoning state propagation.

```
etak = Pose3D(xk_1).oplus(nu_k * self.dt)
```

No compass, landmark or absolute-position measurement is used in this update. Consequently, every encoder error is integrated into the next pose and cannot be removed by the estimator.

4 Results and discussion

Figure 2 compares the simulated trajectory with the pose obtained from dead reckoning. The report figure was generated with the implemented model and commands, using a fixed NumPy seed of 7 so that this draft is reproducible. The first panel contains the positive-yaw-rate command; the second includes both opposite-turn segments.

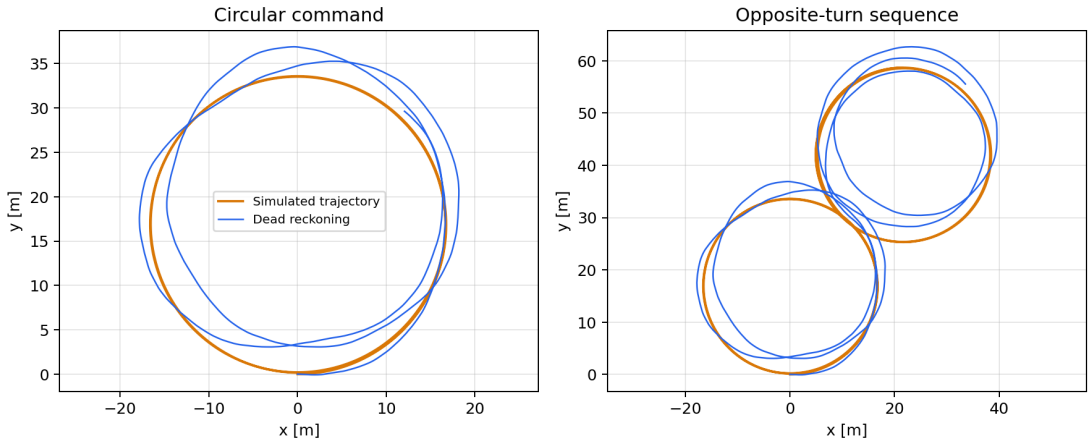


Figure 2: Simulated and dead-reckoning trajectories for the circular and opposite-turn command sequence.

The circular command produces the expected approximately circular path. The dead-reckoning estimate initially follows the simulated trajectory but slowly separates from it as noisy encoder increments are accumulated. In the fixed run shown here, the position error after the first 5000-step segment is 1.019 m. After both 5000-step segments, the final position error is 12.481 m and the final wrapped yaw error is approximately $-50.15°$. The exact values vary between runs because both the motion and encoder models are stochastic.

The larger error over the complete sequence is expected. Dead reckoning is an open-loop estimate: a small angular error rotates every later translation in the wrong direction, so position drift grows with travelled distance and time. This behaviour motivates the compass and map-based corrections used in the subsequent laboratories.

5 Conclusions

The two laboratories establish a complete chain from planar rigid-body kinematics to simulated sensing and odometric localisation. The `Pose3D` class provides consistent direct and inverse compounding, which is then reused by both the simulator and estimator. The differential-drive simulator generates noisy motion and encoder pulses, while the dead-reckoning class converts those pulses into incremental body motion and propagates the pose estimate. Circular and opposite-turn tests verify the expected robot motion and also demonstrate the fundamental limitation of dead reckoning: without an absolute observation, encoder noise produces unbounded drift.

References

[1] P. Ridao, *Probabilistic Robot Localization & Mapping: Probabilistic Robotics Labs*, draft laboratory guide, University of Girona, 2025.

[2] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed., Springer, 2017.